Experimental Study on the Five Sort Algorithms

You Yang, Ping Yu, Yan Gan
School of Computer and Information Science
Chongqing Normal University
Chongqing, 400047, China
youyung@yahoo.com, {40465742, 790147687}@qq.com

Abstract **- Sorting algorithm is one of the most basic research fields in computer science. It's goal is to make record easier to search, insert and delete. Through the description of five sort algorithms: bubble, select, insert, merger and quick, the time and space complexity was summarized. Furthermore, two categories of $O(n^2)$ and $O(n\log n)$ could be found out. From the aspects of input sequence scale and input sequence random degree, some results were obtained based on the experiments. When the size of records is small, insertion sort or selection sort performs well. When the sequence is ordered, insertion sort or bubble sort performs well. When the size of records is large, quick sort or merge sort performs well. Different application could select appropriate sort algorithm according to these rules.**

Keywords - sort algorithm; bubble sort; select sort; insert sort; merger sort; quick sort

I. INTRODUCTION

Sort is an important operation in computer programming. For any sequence of records or data, sort is an ordering procedure by a type of keyword. The sorted sequence is benefit for record searching, insertion and deletion. Thus enhance the efficiency of these operations.

Two categories of sort algorithms were classified according to the records whether stored in the main memory[1]. One category is the internal sort which stores the records in the main memory. Another is the external sort which stores the records in the hard disk because of the records' large space occupation. In fact, by utilizing the splitting and emerging, the external sort could be converted to internal sort. Therefore, only internal sort algorithms such as bubble, select, insertion, emerging and quick sort were discussed bellow.

For the convenience, we make two assumptions bellow. One is the sequence ordering ascending in default. Another is all the records of the sequence were stored in the continuous address memory cells. In this situation, the order of records was determined by the position which stored in the memory. The sort is the move operation of records.

II. FIVE SORT ALGORITHMS

Five sort algorithms were selected to do the experiments. They were the sort algorithms of bubble, select, insert, merge and quick. They were the popular sort algorithms in applications. Because of that there were many variations of these algorithms, therefore the algorithms were defined firstly in this section, then the performances of the algorithms were given by experiments in the next section.

A. Bubble Sort

Bubble sort algorithm[2] used in the experiments below was described by C language as:

```
template<class Type>
void BubbleSort(Type *R,int n)
{int i,j; Type temp;
for(i=0;i<n-1;i++)
  for(j=0;j<n-i-1;j++)
  if(R[j]>R[j+1])
  {temp=R[j];R[j]=R[j+1];R[j+1]=temp;}}
```

In the best case, the input sequence is positive, the algorithm needs $n-1$ comparisons, and its time complexity is $O(n)$. In the worst case, the input sequence is negative, the algorithm needs $\sum_{i=n}^{2}(i-1)=n(n-1)/2$ comparisons, and its time complexity is $O(n^2)$. Whatever, the space complexity of bubble sort is $O(1)$.

B. Select Sort

Select sort algorithm[3] used in the experiments below was described by C language as:

```
template<class Type>
void SelectSort(Type R[], int n)
{int i,j,k;
for(i=1;i<n;i++)
{k=i;for(j=i+1;j<=n;j++)
   if(R[j] <R[k])  k=j;
   if(k!=i){R[0]=R[i];R[i]=R[k];
   R[k]=R[0];}}}
```

It's easy to prove: the algorithm needs $n(n-1)/2$ comparison operations whatever the input sequence is. That is the algorithm's time complexity is $O(n^2)$. Additionally, its space complexity of select sort is $O(1)$.

C. Insert Sort

Insert sort algorithm[4] used in the experiments below was described by C language as:

```
template<class Type>
void
InsertSort(Type R[],int left,int right)
{Type temp;int i,j;
for(i=left+1;i<right;i++)
```

978-1-4244-9439-2/11/$26.00 ©2011 IEEE

```
if(R[i]<R[i-1]){temp=R[i];j=i-1;
do{R[j+1]=R[j];j--;}
while(j>=left&&temp<R[j]);
R[j+1]=temp;}}
```

The time complexity of insert sort algorithm described above is $O(n^2)$ and its space complexity is $O(1)$. When the input sequence is positive, the algorithm needs $n-1$ comparison operations without any move operation. Otherwise, when the input sequence is negative, the algorithm needs $(n+2)(n-1)/2$ comparison operations while $(n+4)(n-1)/2$ move operations needed.

D. Merger Sort

Merger sort algorithm[5,6] used in the experiments below was described by C language as:

```
template<class Type>
void merge(Type R1[],Type R2[],int l,int
m,int r)
{int i=l,j=m+1,k=l;
while((i<=m) && (j<=r))
 if(R1[i]<=R1[j]) R2[k++]=R1[i++];
 else R2[k++]=R1[j++];
if(i>m)
for(int q=j;q<=r;q++) R2[k++]=R1[q];
else for(int q=i;q<=r;q++)
R2[k++]=R1[q];}
void mergesort(Type R[],int left,int
right)
{if(left<right){int i=(left+right)/2;
mergesort(R,left,i);
mergesort(R,i+1,right);int b[M];
merge(R,b,left,i,right);
copy(R,b,left,right);}}
```

The time complexity of merge sort is $O(n\log n)$ and space complexity is $O(1)$. Two operations named *merge* and *copy* could be accomplished in $O(n)$. In the worst case, the algorithm needs $T(n)$ to calculate:

$$T(n)=\begin{cases}O(1), & n\le 1\\ 2T(n/2)+O(n), & otherwise\end{cases} \quad (1)$$

From equation above, we could get $T(n)=O(n\log n)$.

E. Quick Sort

Quick sort algorithm[7,8] used in the experiments below was described by C language as:

```
template<class Type>
int paritition(Type R[],int p,int r)
{int i=p,j=r+1;Type x=R[p],temp;
while(1)
{while(R[++i]<x && i<r);
while(R[--j]>x); if(i>=j)  break;
temp=R[i];R[i]=R[j];R[j]=temp;}
R[p]=R[j];R[j]=x;return j;}
void quicksort(Type R[],int p,int r)
{if(p<r)
{int q=paritition(R,p,r);
 quicksort(R,p,q-1);quicksort(R,q+1,r);}}
```

The time complexity of this quick sort is $O(n^2)$. When the input sequence is ordered (positive or negative), the algorithm needs $\sum_{i=1}^{n-1}(n-i)=O(n^2)$ comparison operations. When the input sequence is random, the time $T(n)$ needed to sort n records satisfied equation (1).

III. Experimental Study

In order to compare the performance of the five sort algorithms above, we use a desktop computer (AMD Sempron Dual Core Processor 2100，1.81GHz，2.87GB RAM, Windows XP operating system) to do a serial experiments. Under vs6.0, using C language, the programs test the performances of these five algorithms from input scale size by utilizing random function calling and time function calling.

A. Experiments and Results

When the input sequence is produced by a random function, and the input scale varied from 2000 to 128000, five sort algorithms' time costs were demonstrated by table 1 and figure 1.

TABLE 1. FIVE SORT ALGORITHMS' TIME COST UNDER DIFFERENT INPUT SCALE

records	bubble	select	insert	merge	quick
2000	0.016	0.032	0.016	0.000	0.000
4000	0.063	0.125	0.047	0.016	0.000
8000	0.219	0.484	0.219	0.031	0.000
16000	0.844	1.921	1.016	0.078	0.000
32000	3.125	7.656	3.219	0.906	0.016
64000	12.000	30.235	14.594	3.703	0.016
128000	48.391	123.344	54.797	50.125	0.032

From the table and the figure above, we got: when the scale of input sequence was small, the difference of time cost between these five algorithms was small. But with the scale of input sequence becoming larger and larger, the difference became lager and larger. Among these algorithms, the quick sort was the best, then the merge sort, traditional bubble sort and insert sort. The worst was the select sort. When the number of the records was greater than 62000, the time cost of the traditional bubble sort and the insert sort was almost the same. From the number of this point, the select sort cost more and more time than other.

Whatever, the time cost curve of quick sort was almost a line. It's the slowest changing with the input scale increasing.

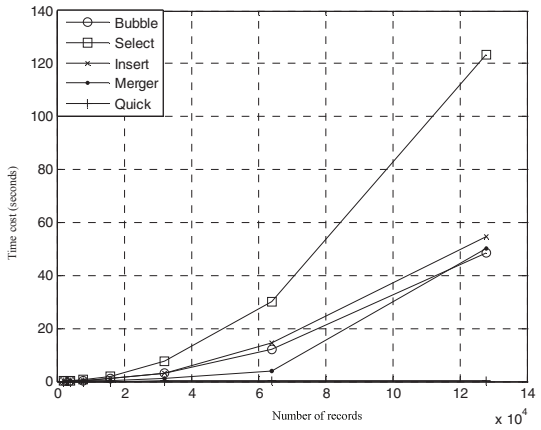


Fig.1 Time cost comparison of five sort algorithms under different input scale

When the input sequence is positive, and the input scale varied from 1000 to 10000, five sort algorithms' time costs were demonstrated by table 2 and figure 2.

TABLE 2. FIVE SORT ALGORITHMS' TIME COST WITH POSITIVE INPUT SEQUENCE

records	bubble	select	insert	merge	quick
1000	0.000	0.000	0.000	0.000	0.000
2000	0.015	0.031	0.000	0.000	0.000
3000	0.032	0.078	0.000	0.000	0.031
4000	0.047	0.125	0.000	0.015	0.047
5000	0.078	0.188	0.000	0.000	0.078
6000	0.125	0.250	0.000	0.015	0.109
7000	0.156	0.344	0.000	0.015	0.156
8000	0.203	0.454	0.000	0.015	0.203
9000	0.265	0.594	0.000	0.015	0.266
10000	0.313	0.719	0.000	0.031	0.312

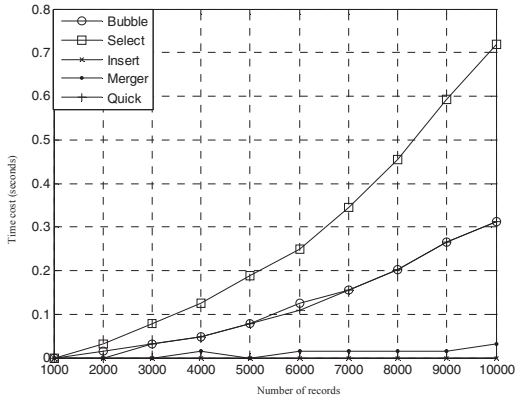


Fig.2 Time cost comparison of five sort algorithms with positive input sequences

When the input sequence is negative, and the input scale varied from 1000 to 10000, five sort algorithms' time costs were demonstrated by table 3 and figure 3.

From table 3 and figure 3, we could get some results bellow. The first result was that the quick sort was the best and the select sort was the worst. The second result was that the time cost curves of the select sort and the traditional bubble sort were almost the same. These two curves were exponential with the input records increasing. The third result was that the time cost curve of quick sort was linear, especially when the number of the input records was less than 5000. The final result was that the curves of the merge sort and insert sort were exponential also, but they were lower sensitive to the input scale increasing than the select's and the traditional bubble's.

TABLE 3. FIVE SORT ALGORITHMS' TIME COST WITH NEGATIVE INPUT SEQUENCE

records	bubble	select	insert	merge	quick
1000	0.000	0.000	0.000	0.000	0.016
2000	0.016	0.031	0.015	0.000	0.016
3000	0.063	0.078	0.032	0.000	0.031
4000	0.109	0.125	0.063	0.000	0.047
5000	0.188	0.188	0.109	0.000	0.078
6000	0.250	0.266	0.156	0.015	0.110
7000	0.344	0.359	0.219	0.015	0.156
8000	0.437	0.485	0.265	0.031	0.203
9000	0.562	0.609	0.359	0.031	0.265
10000	0.719	0.765	0.438	0.031	0.328

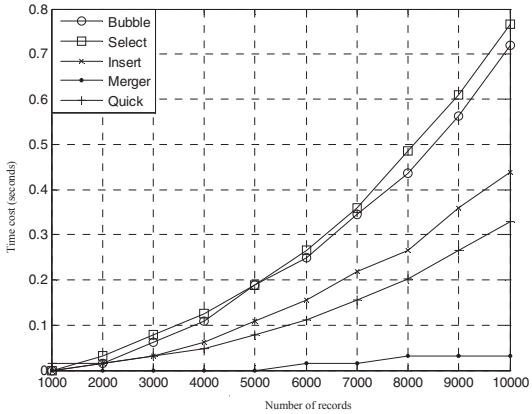


Fig.3 Time cost comparison of five sort algorithms with negative input sequences

B. Performances Evaluation

Two criteria to evaluate the sort algorithms: time and space. The time related to the comparison operations and move operations of records in the algorithms. The space may dependent or independent to the input sequence scale. If the additional space needed in the algorithm is independent to the input, its space complexity is $O(1)$. Otherwise, its space complexity is $O(n)$.

Let N denote the number of input records, in which there are n elements were ordered. Then we could define K, called ordered factor.

$$K = \frac{n}{N} \tag{2}$$

$K \in [0,1]$ reflects the sort degree of a random sequence. K is bigger, more ordered exists in the sequence. Otherwise, K is smaller, more random exists in the sequence. Let KCN represent the number of comparison operation, and RCN represent the number of remove operation, $T(n)$ and $S(n)$ represent the algorithm time complexity and space complexity respectively. When $K \to 1$, then $RCN \to 0$ and $T(n)$ become lesser. When $K \to 0$, then RCN and $T(n)$ become bigger. According to $S(n)$ whether independent or dependent to the input scale, its value is $O(1)$ or $O(n)$.

IV. Conclusion

Based on the analysis above, we could obtain a summative result table below. Two types of the algorithms could be classified. One is the type of time complexity $O(n^2)$. They are the sort algorithms of traditional bubble, select and insert. Another is the type of time complexity $O(n \log n)$. They are the merge sort and the quick sort.

TABLE 4. FIVE SORT ALGORITHMS' TIME COMPLEXITY COMPARISON

Algorithms	Average	Worst	Space
Bubble	$O(n^2)$	$O(n^2)$	$O(1)$
Select	$O(n^2)$	$O(n^2)$	$O(1)$
Insert	$O(n^2)$	$O(n^2)$	$O(1)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick	$O(n \log n)$	$O(n^2)$	$O(\log n)$

From the average time algorithms cost, the quick sort and merge sort are superior to other three algorithms'. But in the worst situation, the quick sort cost too much more time than the merge sort.

When the input scale isn't big, time cost of five sort algorithms has not an obvious difference. But with the input scale increasing, the quick sort has an obvious advantage over other four algorithms.

For the space occupation, the quick sort and merge sort cost two much than others', their space complexity is $O(\log n)$ and $O(n)$, dependent to the input scale. Other three sort algorithms cost little, their space complexity is $O(1)$, independent to the input scale.

For the application, appropriate sort algorithm is selected according to the attributes of input sequence. If the input scale is small, insert and select algorithm is a good choice. If some patterns or rules could be found in the input sequence, insert and bubble sort is a proper choice. But when the input scale is large, merge sort and quick sort is the necessary choice essentially.

Acknowledgment

This work is partially supported by Doctor Fund of Chongqing Normal University (10XLB006) to You Y. and Researching Fund of Chongqing Teaching Bureau (KJ100623) to Ping Y.

References

[1] Knuth D. E. The Art of Computer Programming – Sorting and Searching. Addison Wesley Publishing Company, Inc., 1973, 3:145-158.
[2] Debosiewicz W. An Efficient Variation of Bubble Sort. Information Processing Letters. 1980, 11(1): 5-6.
[3] Iraj H., Afsari M. H. S., Hassanzadeh S. A New External Sorting Algorithm with Selecting the Record List Location. USEAS Transactions on Communications. 2006, 5(5):909-913.
[4] Kumari A., Chakraborty S. Software Complexity: A Statistical Case Study through Insertion Sort. Applied Mathmatics and Computation. 2007, 190(1): 40-50.
[5] Jafarlou M. Z., Fard P. Y. Heuristic and Pattern Based Merge Sort. Procedia Computer Science. 2011, 3: 322-324.
[6] Nardelli E., Proietti G. Efficient Unbalanced Merge-Sort. Information Science. 2006, 176(10):1321-1337.
[7] Feng H. Analysis of the Complexity of Quick Sort for two Dimension Table. Jisuanji Xuebao. In Chinese. 2007, 30(6):963-968.
[8] Yueying P., Shicai L., Miao L. Quick Sorting Algorithm of Matrix. The 8th International Conference on Electronic Measurement and Instruments. Auguest 16-18, 2007. Xian, China. 2007, 2601-2605.